

Fiche mémo Python — Listes

Synthèse concise de la page « Utilisation des Listes en Python » de Stéphane Robert. La fiche regroupe les fonctions, méthodes, opérateurs et syntaxes utiles présentés sur la page.

1. Créer une liste

Fonction / syntaxe	Ce que ça fait	Exemple
<code>[]</code>	Crée une liste vide.	<code>ma_liste = []</code>
<code>list()</code>	Crée une liste à partir d'un itérable.	<code>list(range(5))</code> # <code>[0, 1, 2, 3, 4]</code>
<code>range()</code>	Génère une séquence de nombres, souvent convertie avec <code>list()</code> .	<code>list(range(1, 6))</code> # <code>[1, 2, 3, 4, 5]</code>

2. Accéder / extraire

Fonction / syntaxe	Ce que ça fait	Exemple
<code>[index]</code>	Accède à un élément par sa position. Le premier index est 0.	<code>ma_liste[0]</code>
<code>[-1]</code>	Accède au dernier élément ; les indices négatifs partent de la fin.	<code>ma_liste[-1]</code>
<code>[debut:fin]</code>	Extrait une sous-liste (slicing). La borne fin est exclue.	<code>ma_liste[1:4]</code>
<code>[debut:fin:pas]</code>	Slicing avec un pas ; permet notamment d'inverser.	<code>ma_liste[::-1]</code>

3. Ajouter / supprimer

Fonction / syntaxe	Ce que ça fait	Exemple
<code>append(x)</code>	Ajoute un élément à la fin.	<code>ma_liste.append(10)</code>
<code>insert(i, x)</code>	Insère x à l'index i.	<code>ma_liste.insert(1, 10)</code>
<code>extend(iterable)</code>	Ajoute plusieurs éléments à la fin.	<code>ma_liste.extend([10, 20])</code>
<code>remove(x)</code>	Supprime la première occurrence de x.	<code>ma_liste.remove(10)</code>
<code>pop([i])</code>	Supprime et retourne l'élément à l'index i ; sans i, le dernier.	<code>x = ma_liste.pop()</code>
<code>del</code>	Supprime un élément, une tranche ou une variable.	<code>del ma_liste[2]</code>

4. Modifier

Fonction / syntaxe	Ce que ça fait	Exemple
<code>liste[i] = x</code>	Remplace un élément précis.	<code>ma_liste[1] = 45</code>
<code>liste[a:b] = [...]</code>	Remplace plusieurs éléments via slicing.	<code>ma_liste[1:3] = [35, 55]</code>
<code>liste[i:i] = [...]</code>	Insère plusieurs éléments à une position via slicing.	<code>ma_liste[2:2] = [40, 50]</code>
<code>liste[a:b] = []</code>	Supprime une tranche via slicing.	<code>ma_liste[1:3] = []</code>

5. Informations / recherche

Fonction / syntaxe	Ce que ça fait	Exemple
<code>len(liste)</code>	Retourne le nombre d'éléments.	<code>len(ma_liste)</code>
<code>liste.index(x)</code>	Retourne l'index de la première occurrence de x.	<code>ma_liste.index(20)</code>
<code>x in liste</code>	Teste si x est présent dans la liste.	<code>20 in ma_liste</code>
<code>liste.count(x)</code>	Compte le nombre d'occurrences de x.	<code>ma_liste.count(20)</code>

6. Trier / inverser

Fonction / syntaxe	Ce que ça fait	Exemple
liste.sort()	Trie la liste sur place, croissant par défaut.	<code>ma_liste.sort()</code>
liste.sort(reverse=True)	Trie la liste sur place dans l'ordre décroissant.	<code>ma_liste.sort(reverse=True)</code>
sorted(liste)	Retourne une nouvelle liste triée sans modifier l'originale.	<code>nouvelle = sorted(ma_liste)</code>
liste.reverse()	Inverse la liste sur place ; ne réalise pas un tri.	<code>ma_liste.reverse()</code>

7. Parcourir

Fonction / syntaxe	Ce que ça fait	Exemple
for x in liste	Parcourt directement chaque élément.	<code>for x in ma_liste:</code>
enumerate(liste)	Parcourt les éléments avec leur index.	<code>for i, x in enumerate(ma_liste):</code>
while + len()	Parcourt une liste avec un index et une condition.	<code>i=0\nwhile i < len(ma_liste): ...</code>

8. Copies / fusion

Fonction / syntaxe	Ce que ça fait	Exemple
liste.copy()	Crée une copie superficielle de la liste.	<code>copie = ma_liste.copy()</code>
liste[:]	Autre façon de créer une copie superficielle.	<code>copie = ma_liste[:]</code>
liste1 + liste2	Fusionne deux listes en créant une nouvelle liste.	<code>fusion = liste1 + liste2</code>
copy.deepcopy()	Crée une copie profonde, utile avec les listes imbriquées.	<code>copy.deepcopy(ma_liste)</code>

9. Compréhensions

Fonction / syntaxe	Ce que ça fait	Exemple
[expression for x in liste]	Crée une nouvelle liste en transformant chaque élément.	<code>carres = [x**2 for x in nombres]</code>
[expression for x in liste if condition]	Crée une liste filtrée selon une condition.	<code>pairs = [x for x in nombres if x % 2 == 0]</code>
Compréhension imbriquée	Permet de transformer des listes de listes.	<code>[[x**2 for x in l] for l in liste]</code>

10. map() / filter()

Fonction / syntaxe	Ce que ça fait	Exemple
map(fonction, iterable)	Applique une fonction à chaque élément ; retourne un itérateur.	<code>list(map(lambda x: x**2, ma_liste))</code>
filter(condition, iterable)	Conserve les éléments satisfaisant une condition ; retourne un itérateur.	<code>list(filter(lambda x: x%2==0, ma_liste))</code>
lambda	Crée une petite fonction anonyme, souvent utilisée avec map/filter/sorted.	<code>lambda x: x**2</code>

11. Pièges / bonnes pratiques

Fonction / syntaxe	Ce que ça fait	Exemple
Modifier une liste pendant un for	À éviter : supprimer des éléments pendant le parcours peut provoquer des comportements inattendus.	<code>nouvelle = [x for x in ma_liste if x >= 10]</code>
copy() vs deepcopy()	copy() ne copie pas indépendamment les sous-listes ; deepcopy() oui.	<code>import copy\ncopie = copy.deepcopy(liste)</code>
sort() vs sorted()	sort() modifie la liste ; sorted() crée une nouvelle liste.	<code>tri = sorted(ma_liste)</code>

À retenir

Ajout : append / insert / extend **Suppression** : remove / pop / del **Recherche** : index / in / count **Tri** : sort / sorted **Parcours** :
for / enumerate **Transformation** : compréhension / map / filter.